

Python - Remove Dictionary Items

Remove Dictionary Items

Removing dictionary items in Python refers to deleting key-value pairs from an existing dictionary. Dictionaries are mutable data structures that hold pairs of keys and their associated values. Each key acts as a unique identifier, mapping to a specific value within the dictionary.

Removing items from a dictionary allows you to eliminate unnecessary or unwanted data from the dictionary, thereby reducing its size and modifying its content.

We can remove dictionary items in Python using various ways such as –

- using the del keyword
- using the pop() method
- using the popitem() method
- using the clear() method
- using dictionary comprehension

Remove Dictionary Items Using del Keyword

The del keyword in Python is used to delete objects. In the context of dictionaries, it is used to remove an item or a slice of items from the **dictionary**, based on the specified key(s).

We can remove dictionary items using the del keyword by specifying the key of the item we want to remove. This will delete the key-value pair associated with the specified key from the dictionary.

Example 1

In the following example, we are creating a dictionary named numbers with integer keys and their corresponding string values. Then, delete the item with the key '20' using the del keyword –

```
numbers = {10:"Ten", 20:"Twenty", 30:"Thirty",40:"Forty"}
print ("numbers dictionary before delete operation: \n", numbers)
del numbers[20]
print ("numbers dictionary before delete operation: \n", numbers)
```

It will produce the following output –

```
numbers dictionary before delete operation:
{10: 'Ten', 20: 'Twenty', 30: 'Thirty', 40: 'Forty'}
numbers dictionary before delete operation:
{10: 'Ten', 30: 'Thirty', 40: 'Forty'}
```

Example 2

The del keyword, when used with a dictionary object, removes the dictionary from memory –

```
numbers = {10:"Ten", 20:"Twenty", 30:"Thirty",40:"Forty"}
print ("numbers dictionary before delete operation: \n", numbers)
del numbers
print ("numbers dictionary before delete operation: \n", numbers)
```

Following is the output obtained –

```
numbers dictionary before delete operation:
{10: 'Ten', 20: 'Twenty', 30: 'Thirty', 40: 'Forty'}
Traceback (most recent call last):
  File "C:\Users\mlath\examples\main.py", line 5, in <module>
    print ("numbers dictionary before delete operation: \n", numbers)
```

^ ^ ^ ^ ^ ^ ^ ^

NameError: name 'numbers' is not defined

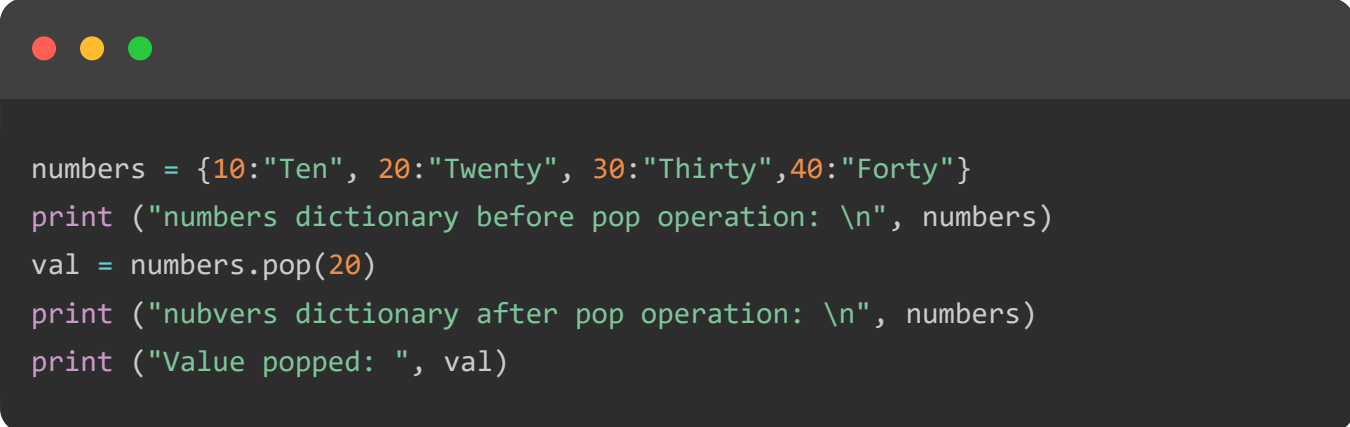
Remove Dictionary Items Using pop() Method

The pop() method in Python is used to remove a specified key from a dictionary and return the corresponding value. If the specified key is not found, it can optionally return a default value instead of raising a KeyError.

We can remove dictionary items using the pop() method by specifying the key of the item we want to remove. This method will return the value associated with the specified key and remove the key-value pair from the dictionary.

Example

In this example, we are using the pop() method to remove the item with the key '20' (storing its value in val) from the 'numbers' dictionary. We then retrieve the updated dictionary and the popped value –



```

numbers = {10:"Ten", 20:"Twenty", 30:"Thirty",40:"Forty"}
print ("numbers dictionary before pop operation: \n", numbers)
val = numbers.pop(20)
print ("nubvers dictionary after pop operation: \n", numbers)
print ("Value popped: ", val)

```

Following is the output of the above code –

```

numbers dictionary before pop operation:
{10: 'Ten', 20: 'Twenty', 30: 'Thirty', 40: 'Forty'}
nubvers dictionary after pop operation:
{10: 'Ten', 30: 'Thirty', 40: 'Forty'}
Value popped:  Twenty

```

Remove Dictionary Items Using popitem() Method

The popitem() method in Python is used to remove and return the last key-value pair from a dictionary.

Since Python 3.7, dictionaries maintain the insertion order, so `popitem()` removes the most recently added item. If the dictionary is empty, calling `popitem()` raises a `KeyError`.

We can remove dictionary items using the `popitem()` method by calling the method on the dictionary, which removes and returns the last key-value pair added to the dictionary.

Example

In the example below, we use the `popitem()` method to remove an arbitrary item from the dictionary 'numbers' (storing both its key-value pair in `val`), and retrieve the updated dictionary along with the popped key-value pair –

```
numbers = {10:"Ten", 20:"Twenty", 30:"Thirty",40:"Forty"}
print ("numbers dictionary before pop operation: \n", numbers)
val = numbers.popitem()
print ("numbers dictionary after pop operation: \n", numbers)
print ("Value popped: ", val)
```

Output of the above code is as shown below –

```
numbers dictionary before pop operation:
{10: 'Ten', 20: 'Twenty', 30: 'Thirty', 40: 'Forty'}
numbers dictionary after pop operation:
{10: 'Ten', 20: 'Twenty', 30: 'Thirty'}
Value popped: (40, 'Forty')
```

Remove Dictionary Items Using `clear()` Method

The `clear()` method in Python is used to remove all items from a dictionary. It effectively empties the dictionary, leaving it with a length of 0.

We can remove dictionary items using the `clear()` method by calling it on the dictionary object. This method removes all key-value pairs from the dictionary, effectively making it empty.

Example

In the following example, we are using the `clear()` method to remove all items from the dictionary 'numbers' –

```
numbers = {10:"Ten", 20:"Twenty", 30:"Thirty",40:"Forty"}
print ("numbers dictionary before clear method: \n", numbers)
numbers.clear()
print ("numbers dictionary after clear method: \n", numbers)
```

We get the output as shown below –

```
numbers dictionary before clear method:
{10: 'Ten', 20: 'Twenty', 30: 'Thirty', 40: 'Forty'}
numbers dictionary after clear method:
{}
```

Remove Dictionary Items Using Dictionary Comprehension

Dictionary comprehension is a concise way to create dictionaries in Python. It follows the same syntax as list comprehension but generates dictionaries instead of lists. With dictionary comprehension, you can iterate over iterable objects (such as lists, tuples, or other dictionaries), apply an expression to each item, and construct key-value pairs based on the result of that expression.

We cannot directly remove dictionary items using dictionary comprehension. Dictionary comprehension is primarily used for creating new dictionaries based on some transformation or filtering of existing data, rather than for removing items from dictionaries.

If you need to remove items from a dictionary based on certain conditions, you would typically use other methods like `del`, `pop()`, or `popitem()`. These methods allow you to explicitly specify which items to remove from the dictionary.

Example

In this example, we remove items 'age' and 'major' from the 'student_info' dictionary based on a predefined list of keys to remove –

```
# Creating a dictionary
student_info = {
    "name": "Alice",
    "age": 21,
    "major": "Computer Science"
}

# Removing items based on conditions
keys_to_remove = ["age", "major"]
for key in keys_to_remove:
    student_info.pop(key, None)

print(student_info)
```

The output obtained is as shown below –

```
{'name': 'Alice'}
```

TOP TUTORIALS

[Python Tutorial](#)

[Java Tutorial](#)

[C++ Tutorial](#)

[C Programming Tutorial](#)

[C# Tutorial](#)

[PHP Tutorial](#)

[R Tutorial](#)

[HTML Tutorial](#)

[CSS Tutorial](#)

[JavaScript Tutorial](#)

[SQL Tutorial](#)

TRENDING TECHNOLOGIES

[Cloud Computing Tutorial](#)

[Amazon Web Services Tutorial](#)

Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler



Chapters ▾

☐ Categories ☰

Online Bash Terminal
Online SQL Compiler
Online Html Editor

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2025. All Rights Reserved.